Camera-to-Printer Frame Transformation

The idea. Print four dots at known printer coordinates. Detect them in the camera image and calculate the camera pose. Reverse that transformation to convert predicted 3D surface points into printer coordinates. Then calculate the nozzle height along the desired path.

1 Overview

There are two operations:

$$\underbrace{\text{Marker} \xrightarrow{\mathbf{R},\mathbf{t}} \text{Camera}}_{\text{3D rotation and translation}} \xrightarrow{\mathbf{K} \text{ and depth division}} \underbrace{\text{Image}}_{\text{2D pixels}} .$$

Since we know each dot's printer XYZ, we can express those points directly in the printer frame, called P. The depth-camera frame is called C.

The figure's focal lengths F_x, F_y and image offsets δ_x, δ_y are written here as f_x, f_y, c_x, c_y. They form the intrinsic matrix

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} . \tag{1}$$

These parameters and lens distortion must already be calibrated for the selected camera stream. The square is used to find the camera's rotation and position relative to the printer.

2 Print and identify the four reference points

For a square of side L at surface height Z_0, the printer coordinates are

$$\begin{aligned} \mathbf{p}_1^P &= (X_0,\ Y_0,\ Z_0)^\mathsf{T}, \\ \mathbf{p}_2^P &= (X_0 + L,\ Y_0,\ Z_0)^\mathsf{T}, \\ \mathbf{p}_3^P &= (X_0 + L,\ Y_0 + L,\ Z_0)^\mathsf{T}, \\ \mathbf{p}_4^P &= (X_0,\ Y_0 + L,\ Z_0)^\mathsf{T}. \end{aligned} \tag{2}$$

Detect the corresponding image locations (u_i, v_i), using the same dot labels. Keep the surface stationary during calibration. Z_0 must describe the observed feature's actual height; it is not automatically the nozzle Z used to print the dot. All XYZ values and translations below use millimetres.

3 Calculate printer-to-camera rotation and translation

Write the rotation and translation explicitly as

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}, \qquad \mathbf{t} = \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} . \tag{3}$$

The forward transformation is rotation followed by translation:

$$\underbrace{\begin{bmatrix} x_C \\ y_C \\ z_C \end{bmatrix}}_{\mathbf{p}_C} = \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}}_{\mathbf{R}} \underbrace{\begin{bmatrix} X \\ Y \\ Z \end{bmatrix}}_{\mathbf{p}_P} + \underbrace{\begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}}_{\mathbf{t}}. \tag{4}$$

Multiplying each row gives the three coordinate equations:

$$\begin{aligned} x_C &= r_{11}X + r_{12}Y + r_{13}Z + t_x, \\ y_C &= r_{21}X + r_{22}Y + r_{23}Z + t_y, \\ z_C &= r_{31}X + r_{32}Y + r_{33}Z + t_z. \end{aligned} \tag{5}$$

The camera projects this point into the image. For distortion-corrected pixels, the complete equation $z_C(u, v, 1)^\mathsf{T} = \mathbf{K}(\mathbf{R}\mathbf{p}_P + \mathbf{t})$ expands to

$$\boxed{z_C \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}.} \tag{6}$$

Dividing by z_C gives the pixel coordinates directly:

$$\boxed{\begin{aligned} u &= f_x \frac{r_{11}X + r_{12}Y + r_{13}Z + t_x}{r_{31}X + r_{32}Y + r_{33}Z + t_z} + c_x, \\ v &= f_y \frac{r_{21}X + r_{22}Y + r_{23}Z + t_y}{r_{31}X + r_{32}Y + r_{33}Z + t_z} + c_y. \end{aligned}} \tag{7}$$

For each reference point, substitute its known X_i, Y_i, Z_i and compare the calculated pixels with the detected u_i, v_i. All four points share the same rotation and translation. The factor z_C is essential: a 3D point must be divided by its depth to obtain a pixel location. A pixel alone does not determine a 3D point.

Pass the four known printer points, their image locations, camera intrinsics, and distortion parameters to a PnP solver. OpenCV's `solvePnP` returns the rotation vector and translation from the supplied object frame to the camera frame [1]. Here that object frame is P:

$$(\texttt{rvec}, \texttt{tvec}) = \text{solvePnP}\big(\{\mathbf{p}_i^P\}, \{(u_i, v_i)\}, \mathbf{K}, \text{distortion}\big). \tag{8}$$

Convert `rvec` into $\mathbf{R}$ using Rodrigues' formula; $\mathbf{t} =$ `tvec`. Use a planar solver appropriate for the point order. Check its pose against measured depth and the known camera orientation, because planar image data can admit competing poses. If dots are detected in a separate RGB camera, also use its calibrated relationship to the depth camera.

4 Reverse the transformation

For printing we need camera-to-printer coordinates. Rearranging Equation (4) gives

$$\boxed{\mathbf{p}_P = \mathbf{R}^\mathsf{T}(\mathbf{p}_C - \mathbf{t}).} \tag{9}$$

The transpose is the inverse of a rotation matrix. The camera-to-printer matrix, written entry by entry, is

$$\boxed{\mathbf{T}_{P\leftarrow C} = \begin{bmatrix} r_{11} & r_{21} & r_{31} & -(r_{11}t_x + r_{21}t_y + r_{31}t_z) \\ r_{12} & r_{22} & r_{32} & -(r_{12}t_x + r_{22}t_y + r_{32}t_z) \\ r_{13} & r_{23} & r_{33} & -(r_{13}t_x + r_{23}t_y + r_{33}t_z) \\ 0 & 0 & 0 & 1 \end{bmatrix}}. \tag{10}$$

It acts on a camera point as

$$\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{T}_{P\leftarrow C} \begin{bmatrix} x_C \\ y_C \\ z_C \\ 1 \end{bmatrix}. \tag{11}$$

Equivalently, the individual printer coordinates are

$$\boxed{\begin{aligned} X &= r_{11}(x_C - t_x) + r_{21}(y_C - t_y) + r_{31}(z_C - t_z), \\ Y &= r_{12}(x_C - t_x) + r_{22}(y_C - t_y) + r_{32}(z_C - t_z), \\ Z &= r_{13}(x_C - t_x) + r_{23}(y_C - t_y) + r_{33}(z_C - t_z). \end{aligned}} \tag{12}$$

5 Transform the predicted surface

The existing height map uses a metric camera XY grid. A valid predicted cell at row i, column j, and future time t_a represents the 3D point

$$\widehat{\mathbf{p}}_C(i,j;t_a) = \begin{bmatrix} x_j \\ y_i \\ \widehat{h}_C(i,j;t_a) \end{bmatrix}, \quad \begin{aligned} x_j &= x_{\min} + (j+\tfrac{1}{2})\Delta, \\ y_i &= y_{\min} + (i+\tfrac{1}{2})\Delta, \end{aligned} \tag{13}$$

where Δ is the cell size and indices start at zero. Transform this complete point using the saved calibration:

$$\boxed{\widehat{\mathbf{p}}_P(i,j;t_a) = \mathbf{R}^{\mathsf{T}} \left(\begin{bmatrix} x_j \\ y_i \\ \widehat{h}_C(i,j;t_a) \end{bmatrix} - \mathbf{t} \right).} \tag{14}$$

Here x_j, y_i are already in millimetres, so no pixel deprojection is needed. Use absolute, unnormalized camera heights. If the model predicts a height change, add its reference height first.

Transform X, Y, and Z together: with a tilted camera, changing camera depth can also change printer X and Y. The transformed points generally form an irregular XY grid. Interpolate or rasterize them onto a printer XY grid to obtain $\widehat{h}_P(X,Y;t_a)$, retaining invalid regions. This assumes the surface has one relevant height at each printer XY location.

6 Calculate the nozzle height

Choose the desired printer path points (X_k, Y_k). Predict the surface for the time t_k when the nozzle will reach each point. For vertical gap g,

$$\boxed{Z_k = \widehat{h}_P(X_k, Y_k; t_k) + g, \qquad \mathbf{p}^P_{\text{nozzle},k} = \begin{bmatrix} X_k \\ Y_k \\ Z_k \end{bmatrix}.} \tag{15}$$

References

[1] OpenCV, *Perspective-n-Point (PnP) pose computation.* https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html